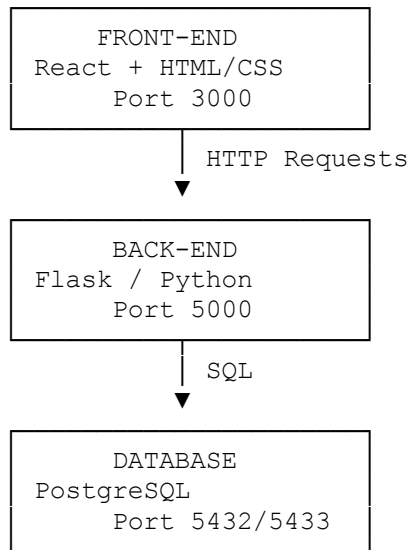


## What we are going to build

We will make a simple **Student Management Application**:



## The 3 tiers

| Tier         | Technology | Purpose            |
|--------------|------------|--------------------|
| Presentation | React      | User interface     |
| Application  | Flask      | API/business logic |
| Data         | PostgreSQL | Store student data |

## Step 1 — Check what is already installed

Open **Command Prompt** and run these commands **one at a time**:

```
python --version
node --version
npm --version
```

For PostgreSQL:

```
"C:\Program Files\PostgreSQL\18\bin\psql.exe" --version
```

Also check whether PostgreSQL is using port 5433:

```
netstat -ano | findstr :5433
```

## Expected result

You should have something similar to:

```
Python 3.x.x
vxx.x.x
10.x.x
psql (PostgreSQL) 18.x
```

Since you've already worked through PostgreSQL/pgAdmin setup and your PostgreSQL port was **5433**, we will use:

PostgreSQL → localhost:5433

rather than assuming the default 5432.

## Step 2 — Create the project folder

Open **Command Prompt** and run:

```
cd %USERPROFILE%\Desktop
```

Then:

```
mkdir three-tier-student-app
```

Then:

```
cd three-tier-student-app
```

Now create the three tiers:

```
mkdir backend
mkdir frontend
```

Your structure will be:

```
three-tier-student-app
├── backend
└── frontend
```

## Step 3 — Create the PostgreSQL database

Open **pgAdmin**.

Go to:

**Servers → PostgreSQL 18 → Databases**

Right-click **Databases** → **Create** → **Database**

Use:

Database: studentdb2

Then click **Save**.

## Step 4 — Create the students table

In pgAdmin, select:

**studentdb2 → Query Tool**

Run:

```
CREATE TABLE students (  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    email VARCHAR(100) NOT NULL,  
    course VARCHAR(100) NOT NULL  
);
```

Then click the **Execute ►** button.

You should get:

Query returned successfully

Then insert some sample data:

```
INSERT INTO students (name, email, course)  
VALUES  
( 'Rahul', 'rahul@gmail.com', 'BCA'),  
( 'Priya', 'priya@gmail.com', 'BSc CS'),  
( 'Amit', 'amit@gmail.com', 'BCA');
```

Run it.

Now your database tier is ready:

```
studentdb2  
├── students  
│   ├── id  
│   ├── name  
│   ├── email  
│   └── course
```

Database tier is complete.

Now we will build the **Backend Tier using Flask** and connect it to your PostgreSQL database **studentdb2** on **port 5433**.

## Step 5 — Create the Flask backend

Open **Command Prompt**.

If you closed it, go back to your project:

```
cd %USERPROFILE%\Desktop\three-tier-student-app
```

Enter the backend folder:

```
cd backend
```

### 5.1 Create a virtual environment

Run:

```
python -m venv venv
```

Activate it:

```
venv\Scripts\activate
```

You should now see something like:

```
(venv) C:\Users\192\Desktop\three-tier-student-app\backend>
```

## 5.2 Install Flask and PostgreSQL connector

Run:

```
python -m pip install flask flask-cors "psycopg[binary]"
```

Wait until the installation finishes.

Then verify Flask:

```
python -c "import flask, psycopg; print('Flask and PostgreSQL connector OK')"
```

You should see:

```
Flask and PostgreSQL connector OK
```

---

## Step 6 — Create `app.py`

Inside the `backend` folder, create a file named:

```
app.py
```

Open the entire project folder in VS Code:

```
code .
```

If code is not recognized, simply open VS Code manually and open:

```
Desktop  
→ three-tier-student-app  
→ backend
```

Put this code inside `app.py`:

```
from flask import Flask, jsonify, request  
from flask_cors import CORS  
import psycopg
```

```
app = Flask(__name__)  
CORS(app)
```

```
# PostgreSQL connection  
DB_CONFIG = {  
    "host": "localhost",  
    "port": 5433,  
    "dbname": "studentdb2",  
    "user": "postgres",  
    "password": "YOUR_POSTGRES_PASSWORD"  
}
```

```
def get_connection():  
    return psycopg.connect(**DB_CONFIG)
```

```

@app.route("/")
def home():
    return jsonify({
        "message": "Student Management API is running"
    })

@app.route("/students", methods=["GET"])
def get_students():
    conn = get_connection()
    cur = conn.cursor()

    cur.execute(
        "SELECT id, name, email, course FROM students ORDER BY id"
    )

    students = cur.fetchall()

    cur.close()
    conn.close()

    result = []

    for student in students:
        result.append({
            "id": student[0],
            "name": student[1],
            "email": student[2],
            "course": student[3]
        })

    return jsonify(result)

@app.route("/students", methods=["POST"])
def add_student():
    data = request.get_json()

    name = data.get("name")
    email = data.get("email")
    course = data.get("course")

    conn = get_connection()
    cur = conn.cursor()

    cur.execute(
        """
        INSERT INTO students (name, email, course)
        VALUES (%s, %s, %s)
        RETURNING id
        """,
        (name, email, course)
    )

    student_id = cur.fetchone()[0]

    conn.commit()

```

```
cur.close()
conn.close()

return jsonify({
    "message": "Student added successfully",
    "id": student_id
}), 201

if __name__ == "__main__":
    app.run(debug=True, port=5000)
```

## Important: change the password

Find this line:

```
"password": "YOUR_POSTGRES_PASSWORD"
```

Replace `YOUR_POSTGRES_PASSWORD` with the password you created when installing PostgreSQL.

For example, if your PostgreSQL password is `postgres123`:

```
"password": "postgres123"
```

Do **not** change these:

```
"host": "localhost",
"port": 5433,
"dbname": "studentdb2",
"user": "postgres",
```

because they match your current setup.

---

## Step 7 — Start the Flask backend

Go back to Command Prompt.

Make sure you are still inside:

```
three-tier-student-app\backend
and that (venv) is visible.
```

Run:

```
python app.py
```

You should see approximately:

```
* Serving Flask app 'app'
* Debug mode: on
* Running on http://127.0.0.1:5000
```

### Test the backend

Open your browser and visit:

<http://127.0.0.1:5000/>

You should get:

```
{
  "message": "Student Management API is running"
}
```

Then open:

<http://127.0.0.1:5000/students>

You should see your students:

```
[
  {
    "id": 1,
    "name": "Rahul",
    "email": "rahul@gmail.com",
    "course": "BCA"
  },
  {
    "id": 2,
    "name": "Priya",
    "email": "priya@gmail.com",
    "course": "BSc CS"
  },
  {
    "id": 3,
    "name": "Amit",
    "email": "amit@gmail.com",
    "course": "BCA"
  }
]
```

**Your application now looks like this**

React Frontend

↓

Flask API

↓

PostgreSQL  
studentdb2

↓

students table

Your PostgreSQL connector is fixed.

Now let's **test the Flask backend connection to PostgreSQL** before creating the React frontend.

## Step 8 — Start Flask

In the same Command Prompt, make sure you see `(venv)` and run:

```
python app.py
```

You should see:

```
* Serving Flask app 'app'
* Debug mode: on
* Running on http://127.0.0.1:5000
```

## Step 9 — Test the Flask API

Keep that Command Prompt **running**.

Open your browser and go to:

<http://127.0.0.1:5000/>

You should see:

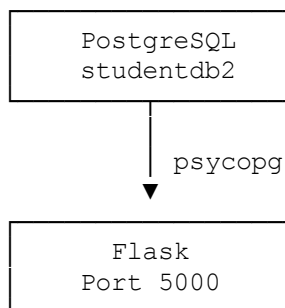
```
{  
  "message": "Student Management API is running"  
}
```

Then open:

<http://127.0.0.1:5000/students>

You should see the students you inserted earlier.

**If `/students` works. Your Backend + Database tiers are successfully connected:**



**Do this now and tell me what you see at `/students`.** Then we'll build the **React frontend (Tier 1)**.

Perfect! 🎉 Your **Flask backend and PostgreSQL database are working correctly**.

Now we build the **Frontend Tier using React**.

## Step 10 — Create the React frontend

**Keep the Flask Command Prompt running.** Open a new **Command Prompt** window.

Go to your project folder:

```
cd %USERPROFILE%\Desktop\three-tier-student-app
```

Now create the React application:

```
npm create vite@latest frontend -- --template react
```

If it asks:

```
Need to install the following packages...
```

```
Ok to proceed? (y)
```

Type:

```
y
```

Wait until it finishes.

## Step 11 — Install React dependencies

Run:

```
cd frontend
```

Then:

```
npm install
```

Then:

```
npm install axios
```

---

## Step 12 — Replace the React code

Open the project in VS Code:

```
code .
```

In VS Code, open:

```
frontend
├── src
│   └── App.jsx
```

Delete everything in App.jsx and paste:

```
import { useEffect, useState } from "react";
import axios from "axios";
import "./App.css";

function App() {
  const [students, setStudents] = useState([]);

  useEffect(() => {
    axios
      .get("http://127.0.0.1:5000/students")
      .then((response) => {
        setStudents(response.data);
      })
      .catch((error) => {
        console.error("Error fetching students:", error);
      });
  }, []);

  return (
    <div className="container">
      <h1>Student Management System</h1>

      <table>
        <thead>
          <tr>
            <th>ID</th>
            <th>Name</th>
            <th>Email</th>
            <th>Course</th>
          </tr>
        </thead>

        <tbody>
          {students.map((student) => (
```

```

        <tr key={student.id}>
          <td>{student.id}</td>
          <td>{student.name}</td>
          <td>{student.email}</td>
          <td>{student.course}</td>
        </tr>
      )})
    </tbody>
  </table>
</div>
);
}

export default App;

```

---

## Step 13 — Add simple CSS

Open:

```

frontend
├── src
│   └── App.css

```

Delete its existing contents and paste:

```

body {
  font-family: Arial, sans-serif;
  background: #f2f2f2;
  margin: 0;
  padding: 40px;
}

.container {
  max-width: 900px;
  margin: auto;
  background: white;
  padding: 30px;
  border-radius: 10px;
}

h1 {
  text-align: center;
}

table {
  width: 100%;
  border-collapse: collapse;
  margin-top: 25px;
}

th,
td {
  border: 1px solid #ccc;
  padding: 12px;
  text-align: left;
}

```

```
th {  
  background: #eeeeee;  
}
```

Save both files.

---

## Step 14 — Start React

Go back to your **second Command Prompt**.

Make sure you are inside:

```
three-tier-student-app\frontend
```

Run:

```
npm run dev
```

You should get something similar to:

```
VITE ready
```

```
Local: http://localhost:5173/
```

Open that address in your browser:

<http://localhost:5173/>



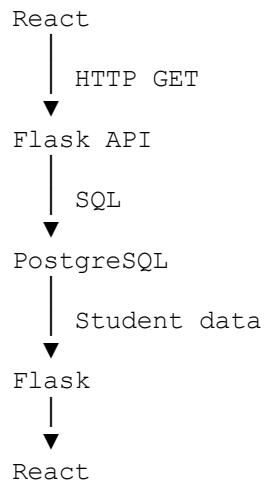
### Expected result

You should see:

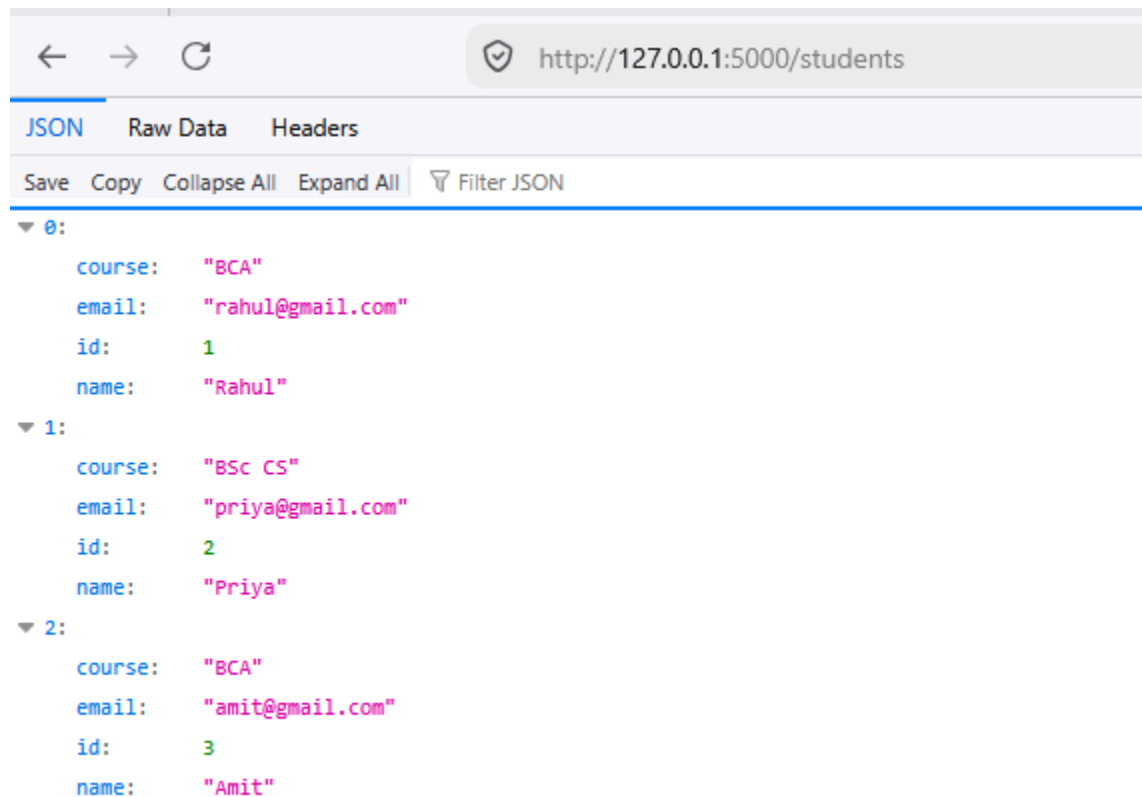
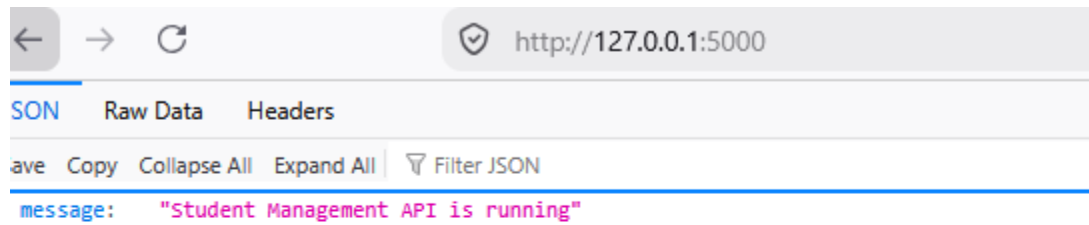
| Student Management System |       |                 |        |
|---------------------------|-------|-----------------|--------|
| ID                        | Name  | Email           | Course |
| -----                     |       |                 |        |
| 1                         | Rahul | rahul@gmail.com | BCA    |
| 2                         | Priya | priya@gmail.com | BSc CS |
| 3                         | Amit  | amit@gmail.com  | BCA    |

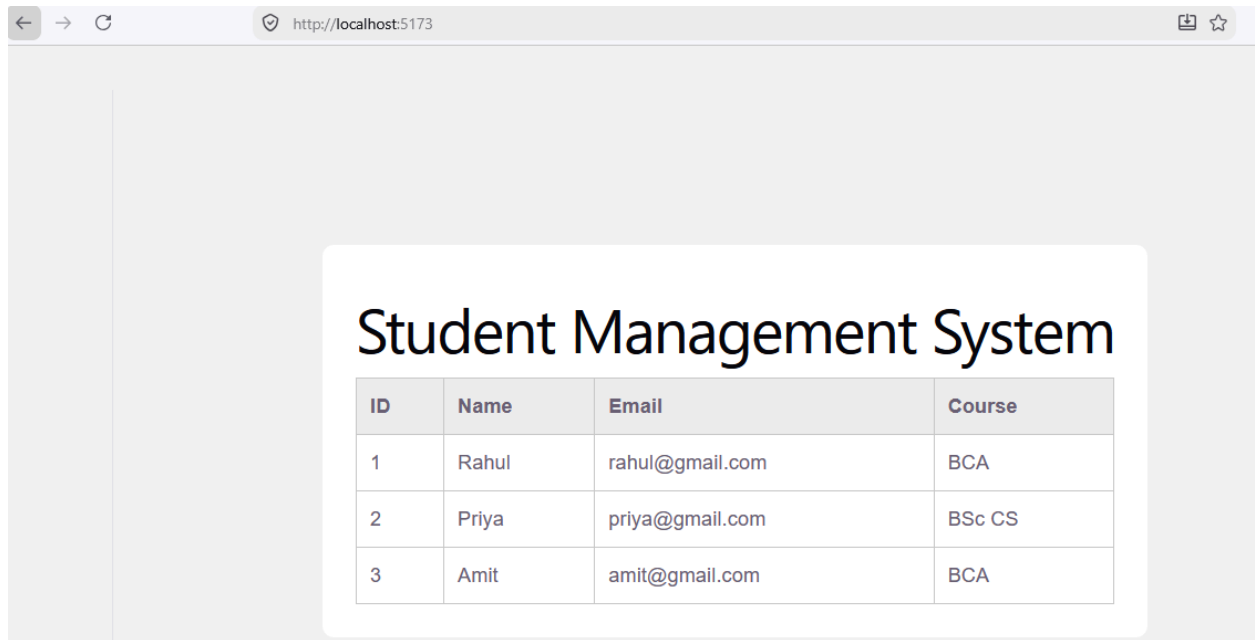
The important part is that these students are **not hard-coded in React**.

The data is flowing:



That means your **3-tier architecture is working**. ✓





```
Select C:\WINDOWS\system32\cmd.exe
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

C:\Users\192>python --version
Python 3.14.7

C:\Users\192>node --version
v24.19.0

C:\Users\192>npm --version
11.17.0

C:\Users\192>"C:\Program Files\PostgreSQL\18\bin\psql.exe" --version
psql (PostgreSQL) 18.6

C:\Users\192>netstat -ano | findstr :5433
TCP    0.0.0.0:5433          0.0.0.0:0           LISTENING        7660
TCP    [::]:5433           [::]:0              LISTENING        7660
```

```
C:\Users\192>cd %USERPROFILE%\Desktop

C:\Users\192\Desktop>mkdir three-tier-student-app

C:\Users\192\Desktop>cd three-tier-student-app

C:\Users\192\Desktop\three-tier-student-app>mkdir backend

C:\Users\192\Desktop\three-tier-student-app>mkdir frontend

C:\Users\192\Desktop\three-tier-student-app>cd %USERPROFILE%\Desktop\three-tier-student-app

C:\Users\192\Desktop\three-tier-student-app>cd backend

C:\Users\192\Desktop\three-tier-student-app\backend>python -m venv venv

C:\Users\192\Desktop\three-tier-student-app\backend>venv\Scripts\activate
```

```
C:\Users\192\Desktop\three-tier-student-app\backend>venv\Scripts\activate

(venv) C:\Users\192\Desktop\three-tier-student-app\backend>python -m pip install flask flask-cors "psycpg[binary]"
Collecting flask
  Using cached flask-3.1.3-py3-none-any.whl.metadata (3.2 kB)
Collecting flask-cors
  Using cached flask_cors-6.0.5-py3-none-any.whl.metadata (5.4 kB)
Collecting psycpg[binary]
  Downloading psycpg-3.3.4-py3-none-any.whl.metadata (4.3 kB)
Collecting blinker>=1.9.0 (from flask)
  Using cached blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
Collecting click>=8.1.3 (from flask)
  Using cached click-8.4.2-py3-none-any.whl.metadata (2.6 kB)
Collecting itsdangerous>=2.2.0 (from flask)
  Using cached itsdangerous-2.2.0-py3-none-any.whl.metadata (1.9 kB)
Collecting jinja2>=3.1.2 (from flask)
  Using cached jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
```

```
ask-cors-6.0.5 itsdangerous-2.2.0 jinja2-3.1.6 markupsafe-3.0.3 psycpg-3.3.4
psycpg-binary-3.3.4 tzdata-2026.3 werkzeug-3.1.8

(venv) C:\Users\192\Desktop\three-tier-student-app\backend>python -m pip insta
ll --upgrade pip
Requirement already satisfied: pip in .\venv\Lib\site-packages (26.2.1)

(venv) C:\Users\192\Desktop\three-tier-student-app\backend>python -m pip insta
ll "psycpg[binary]"
Requirement already satisfied: psycpg[binary] in .\venv\Lib\site-packages (3.
3.4)
Requirement already satisfied: tzdata in .\venv\Lib\site-packages (from psycpg
[binary]) (2026.3)
Requirement already satisfied: psycpg-binary==3.3.4 in .\venv\Lib\site-packag
es (from psycpg[binary]) (3.3.4)

(venv) C:\Users\192\Desktop\three-tier-student-app\backend>cd %USERPROFILE%\De
sktop\three-tier-student-app
```

```
(venv) C:\Users\192\Desktop\three-tier-student-app\backend>cd %USERPROFILE%\De
sktop\three-tier-student-app

(venv) C:\Users\192\Desktop\three-tier-student-app>npm create vite@latest fron
tend -- --template react

> npx
> create-vite frontend --template react

> Which linter to use?
  Oxlint

> Install with npm and start now?
  Yes

> Scaffolding project in C:\Users\192\Desktop\three-tier-student-app\frontend
...
```

```
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

C:\Users\192>cd desktop

C:\Users\192\Desktop>cd %USERPROFILE%\Desktop\three-tier-student-app

C:\Users\192\Desktop\three-tier-student-app>cd frontend

C:\Users\192\Desktop\three-tier-student-app\frontend>npm install

up to date, audited 25 packages in 1s

9 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

```
added 27 packages, and audited 52 packages in 6s

15 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

C:\Users\192\Desktop\three-tier-student-app\frontend>code .

C:\Users\192\Desktop\three-tier-student-app\frontend>npm run dev

> frontend@0.0.0 dev
> vite

10:14:21 am [vite] (client) Re-optimizing dependencies because lockfile has changed
Port 5173 is in use, trying another one...
```

